Agentic AI Layer - Technical Specification

Document Information

- **Version**: 1.0
- **Last Updated**: November 2025
- **Status**: Design Phase
- **Owner**: Engineering Team

---

1. Executive Summary

This specification defines the architecture and implementation details for the Agentic AI layer of the Gaming Companion platform. The agentic layer enables autonomous, multi-step task execution for complex gaming analysis, research, and assistance tasks.

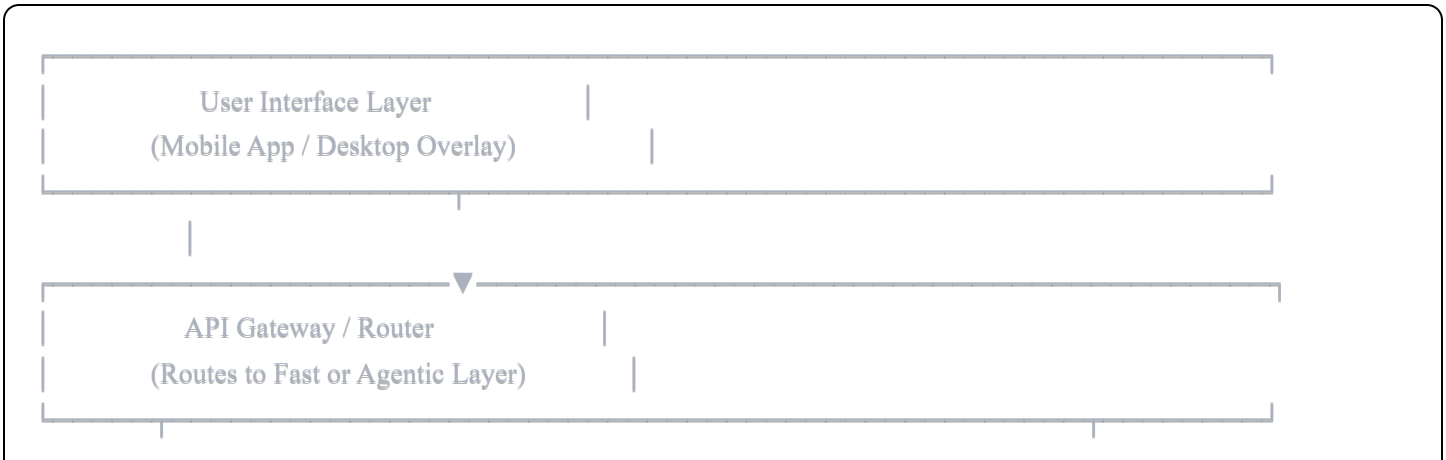
Key Goals

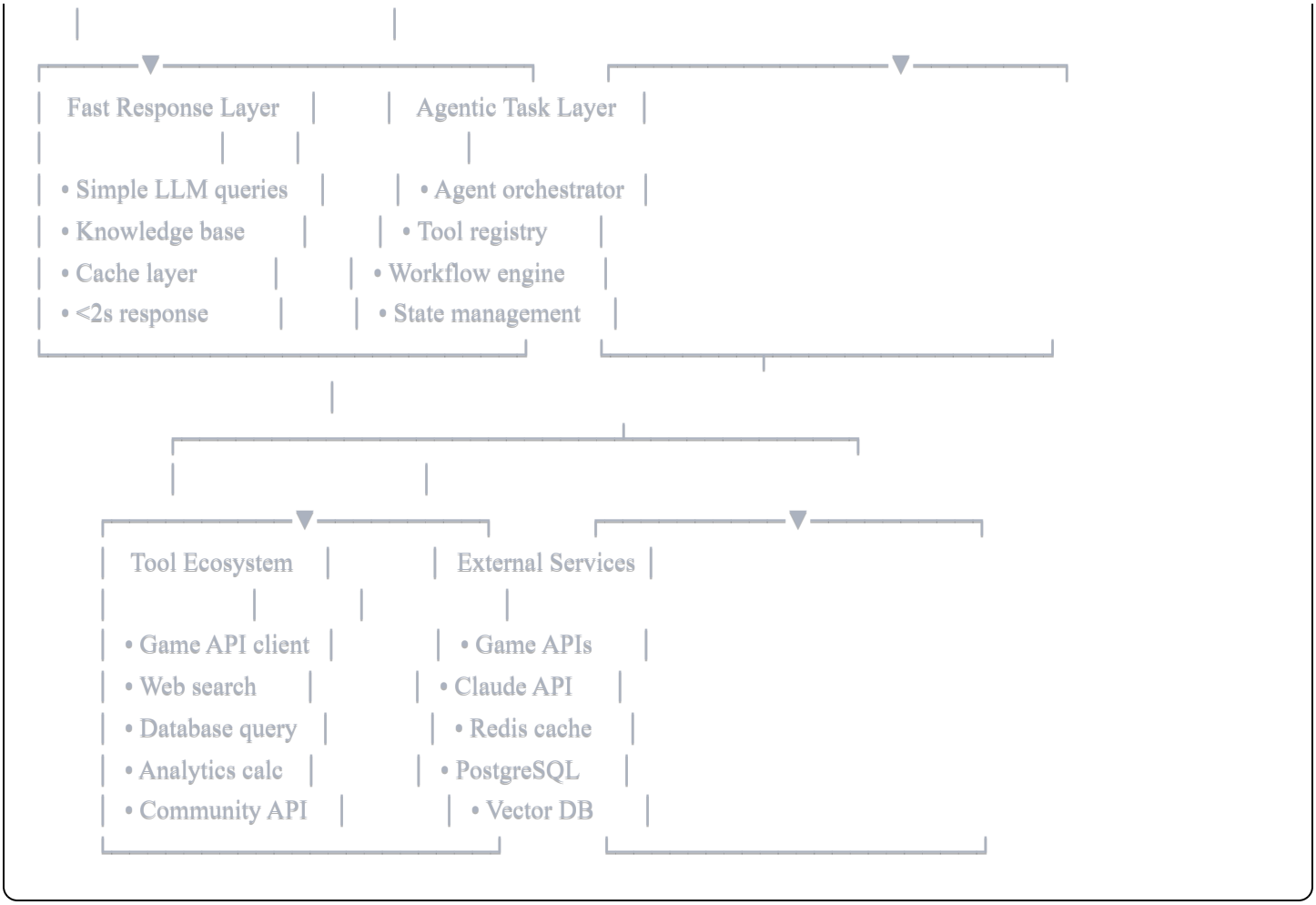
- Enable complex, multi-step reasoning and task execution
- Maintain sub-30 second response times for agentic tasks
- Support extensible tool ecosystem
- Provide reliable fallback mechanisms
- Optimize costs while delivering value

---

2. System Architecture

2.1 High-Level Architecture





2.2 Component Responsibilities

API Gateway / Router

- **Purpose**: Intelligently route requests to appropriate layer
- **Decision Logic**:
  - Real-time queries → Fast layer
  - Complex analysis/research → Agentic layer
  - User explicitly requests deep analysis → Agentic layer
- **Technologies**: Node.js + Express or Python + FastAPI

Fast Response Layer

- **Purpose**: Handle latency-sensitive queries
- **Response Time**: <2 seconds
- **Use Cases**: In-game tips, quick facts, simple Q&A
- **Technologies**:

- LLM: Claude Haiku or GPT-3.5 Turbo (low latency)
- Cache: Redis for frequent queries
- Knowledge Base: Vector database (Pinecone, Weaviate)

Agentic Task Layer

- **Purpose**: Execute complex, multi-step autonomous tasks
- **Response Time**: 10-30 seconds
- **Use Cases**: Match analysis, strategy research, meta analysis
- **Technologies**:
  - LLM: Claude Sonnet or GPT-4 (reasoning capability)
  - Orchestration: LangGraph, CrewAI, or custom framework
  - State: Redis for task state management

---

3. Agent Architecture

3.1 Agent Design Pattern

We'll use the **ReAct (Reasoning + Acting) pattern** with tool use:

```python
```

```python
class GamingAgent:
    """
    Autonomous agent for complex gaming tasks
    """

    def __init__(self, llm_client, tools, max_iterations=10):
        self.llm = llm_client
        self.tools = tools
        self.max_iterations = max_iterations
        self.conversation_history = []

    async def execute_task(self, task_description, context):
        """
        Main agent loop - Reasoning and Acting
        """
        for iteration in range(self.max_iterations):
            # REASONING: Agent thinks about next step
            thought = await self.reason(task_description, context)

            # Check if task is complete
            if thought.is_complete:
                return thought.final_answer

            # ACTING: Agent selects and uses tools
            if thought.action:
                tool_result = await self.use_tool(
                    thought.action.tool_name,
                    thought.action.parameters
                )

                # Update context with results
                context.add_observation(tool_result)

        # Fallback if max iterations reached
        return await self.generate_partial_answer(context)
```

3.2 Agent Types

3.2.1 Analysis Agent

Purpose: Deep dive into player performance

Capabilities:

Example Workflow:

```
User: "Analyze my last 20 ranked games"

Agent Reasoning:
1. Fetch match data → [uses game_api_tool]
2. Calculate KDA, win rate, champion performance
3. Identify weak areas → [uses stats_calculator_tool]
4. Search for improvement strategies → [uses web_search_tool]
5. Generate comprehensive report → [synthesizes findings]
```

3.2.2 Research Agent

Purpose: Meta analysis and strategy research

Capabilities:

- Search patch notes and game updates
- Analyze pro play and high-ELO trends
- Compare builds and strategies
- Synthesize information from multiple sources

Example Workflow:

```
User: "What's the current meta for ADC in League of Legends?"

Agent Reasoning:
1. Search latest patch notes → [uses web_search_tool]
2. Query high-ELO match data → [uses game_api_tool]
3. Search pro play trends → [uses web_search_tool]
4. Analyze champion win rates → [uses database_query_tool]
5. Synthesize meta report → [generates response]
```

3.2.3 Strategy Agent

Purpose: Personalized strategy development

Capabilities:

- Analyze opponent playstyles
- Create counter-strategies
- Develop practice plans
- Set and track improvement goals

---

4. Tool Ecosystem

4.1 Tool Architecture

python

```python
class Tool:
    """Base class for all agent tools"""

    name: str
    description: str
    parameters: dict

    async def execute(self, **kwargs) -> ToolResult:
        """Execute the tool with given parameters"""
        raise NotImplementedError

    def get_schema(self) -> dict:
        """Return tool schema for LLM function calling"""
        return {
            "name": self.name,
            "description": self.description,
            "parameters": self.parameters
        }

class ToolResult:
    """Standardized tool result format"""

    success: bool
    data: dict
    error: Optional[str]
    metadata: dict  # execution time, API calls used, etc.
```

4.2 Core Tools

4.2.1 Game API Tool

python

```python
class GameAPITool(Tool):
    """
    Fetch data from game APIs (Riot, Steam, etc.)
    """

    name = "game_api"
    description = "Fetch player stats, match history, and game data"
    parameters = {
        "game": "string (lol, valorant, dota2, etc.)",
        "endpoint": "string (match_history, player_stats, etc.)",
        "player_id": "string",
        "params": "object (additional API parameters)"
    }

    async def execute(self, game, endpoint, player_id, params=None):
        # Rate limiting
        await self.rate_limiter.acquire()

        # Get appropriate API client
        client = self.get_client(game)

        # Make API call
        try:
            data = await client.fetch(endpoint, player_id, params)
            return ToolResult(
                success=True,
                data=data,
                metadata={"api_calls": 1, "execution_time": 0.5}
            )
        except Exception as e:
            return ToolResult(
                success=False,
                error=str(e),
                data={}
            )
```

4.2.2 Web Search Tool

python

```python
class WebSearchTool(Tool):
    """
    Search the web for game strategies, guides, and meta information
    """

    name = "web_search"
    description = "Search web for gaming strategies, patch notes, and meta"
    parameters = {
        "query": "string",
        "num_results": "integer (default: 5)",
        "filters": "object (date_range, domains, etc.)"
    }

    async def execute(self, query, num_results=5, filters=None):
        # Use web search API (Google, Bing, or custom crawler)
        results = await self.search_engine.search(
            query,
            num_results,
            filters
        )

        # Extract relevant content
        processed_results = []
        for result in results:
            content = await self.fetch_and_extract(result.url)
            processed_results.append({
                "url": result.url,
                "title": result.title,
                "content": content,
                "relevance_score": result.score
            })

        return ToolResult(
            success=True,
            data={"results": processed_results},
            metadata={"sources": len(processed_results)}
        )
```

4.2.3 Database Query Tool

python

```python
class DatabaseQueryTool(Tool):
    """
    Query internal database for player history and analytics
    """

    name = "database_query"
    description = "Query player profiles, historical data, and analytics"
    parameters = {
        "query_type": "string (player_stats, match_history, goals, etc.)",
        "user_id": "string",
        "filters": "object (date_range, game_type, etc.)"
    }

    async def execute(self, query_type, user_id, filters=None):
        # Build and execute database query
        query = self.build_query(query_type, user_id, filters)
        results = await self.db.execute(query)

        return ToolResult(
            success=True,
            data={"results": results},
            metadata={"rows_returned": len(results)}
        )
```

4.2.4 Analytics Calculator Tool

```python
```

```python
class AnalyticsCalculatorTool(Tool):
    """
    Perform complex statistical calculations on game data
    """

    name = "analytics_calculator"
    description = "Calculate advanced statistics and trends"
    parameters = {
        "calculation_type": "string (kda_trend, win_rate, etc.)",
        "data": "array (match data to analyze)",
        "comparison_baseline": "object (optional baseline for comparison)"
    }

    async def execute(self, calculation_type, data, comparison_baseline=None):
        # Perform calculation
        calculator = self.get_calculator(calculation_type)
        results = calculator.calculate(data, comparison_baseline)

        return ToolResult(
            success=True,
            data={"analytics": results},
            metadata={"data_points": len(data)}
        )
```

4.2.5 Knowledge Base Query Tool

python

```python
class KnowledgeBaseTool(Tool):
    """
    Query vector database for game knowledge and guides
    """

    name = "knowledge_base"
    description = "Search curated game knowledge base"
    parameters = {
        "query": "string",
        "game": "string",
        "category": "string (champions, items, strategies, etc.)"
    }

    async def execute(self, query, game, category=None):
        # Vector search in knowledge base
        embedding = await self.embeddings.embed(query)
        results = await self.vector_db.search(
            embedding,
            filters={"game": game, "category": category}
        )

        return ToolResult(
            success=True,
            data={"knowledge": results},
            metadata={"relevance_scores": [r.score for r in results]}
        )
```

4.3 Tool Registry

python

```python
class ToolRegistry:
    """
    Centralized registry for all available tools
    """

    def __init__(self):
        self.tools = {}
        self.register_default_tools()

    def register(self, tool: Tool):
        """Register a new tool"""
        self.tools[tool.name] = tool

    def get_tool(self, name: str) -> Tool:
        """Get tool by name"""
        return self.tools.get(name)

    def get_all_schemas(self) -> list:
        """Get all tool schemas for LLM function calling"""
        return [tool.get_schema() for tool in self.tools.values()]

    def register_default_tools(self):
        """Register core tools"""
        self.register(GameAPITool())
        self.register(WebSearchTool())
        self.register(DatabaseQueryTool())
        self.register(AnalyticsCalculatorTool())
        self.register(KnowledgeBaseTool())
```

---

5. Workflow Engine

5.1 Task State Management

python

```python
class TaskState:
    """
    Maintains state throughout agent execution
    """

    def __init__(self, task_id: str, user_id: str):
        self.task_id = task_id
        self.user_id = user_id
        self.status = "running"
        self.steps = []
        self.context = {}
        self.created_at = datetime.now()
        self.updated_at = datetime.now()

    def add_step(self, step_type: str, data: dict):
        """Record a step in the workflow"""
        self.steps.append({
            "type": step_type,
            "data": data,
            "timestamp": datetime.now()
        })
        self.updated_at = datetime.now()

    def update_context(self, key: str, value: any):
        """Update task context"""
        self.context[key] = value
        self.updated_at = datetime.now()

    async def save(self):
        """Persist state to Redis"""
        await redis.set(
            f"task:{self.task_id}",
            json.dumps(self.to_dict()),
            ex=3600  # 1 hour expiry
        )
```

5.2 Workflow Orchestration

python

```python
class WorkflowOrchestrator:
    """
    Orchestrates complex multi-agent workflows
    """

    async def execute_workflow(
        self,
        workflow_type: str,
        task_description: str,
        user_context: dict
    ) -> WorkflowResult:
        """
        Execute a predefined workflow
        """
        # Create task state
        task_state = TaskState(
            task_id=generate_uuid(),
            user_id=user_context["user_id"]
        )

        # Get workflow definition
        workflow = self.get_workflow(workflow_type)

        try:
            # Execute workflow steps
            for step in workflow.steps:
                result = await self.execute_step(
                    step,
                    task_state,
                    user_context
                )

                task_state.add_step(step.type, result)

                # Check for early termination
                if step.is_terminal and result.should_stop:
                    break

            # Generate final response
            final_result = await self.synthesize_results(task_state)

            task_state.status = "completed"
            await task_state.save()
```

```
            return WorkflowResult(
                success=True,
                data=final_result,
                task_id=task_state.task_id
            )

        except Exception as e:
            task_state.status = "failed"
            task_state.error = str(e)
            await task_state.save()

            # Return partial results if available
            return WorkflowResult(
                success=False,
                error=str(e),
                partial_data=self.get_partial_results(task_state)
            )
```

5.3 Predefined Workflows

Match Analysis Workflow

python

```
class MatchAnalysisWorkflow(Workflow):
    """
    Workflow for analyzing player's recent matches
    """

    steps = [
        WorkflowStep(
            name="fetch_matches",
            tool="game_api",
            params_template={
                "endpoint": "match_history",
                "num_matches": 20
            }
        ),
        WorkflowStep(
            name="calculate_stats",
            tool="analytics_calculator",
            params_template={
                "calculation_type": "comprehensive_analysis"
            },
            depends_on=["fetch_matches"]
        ),
        WorkflowStep(
            name="identify_patterns",
            tool="analytics_calculator",
            params_template={
                "calculation_type": "pattern_detection"
            },
            depends_on=["calculate_stats"]
        ),
        WorkflowStep(
            name="search_improvement_tips",
            tool="web_search",
            params_template={
                "query": "improvement strategies for {weak_areas}"
            },
            depends_on=["identify_patterns"]
        ),
        WorkflowStep(
            name="generate_report",
            is_terminal=True
```

```
    )
]
```

---

6. LLM Integration

6.1 Provider Abstraction

python

```python
class LLMProvider:
    """Abstract base class for LLM providers"""

    async def complete(
        self,
        messages: list,
        tools: list = None,
        max_tokens: int = 4000
    ) -> LLMResponse:
        raise NotImplementedError

class ClaudeProvider(LLMProvider):
    """Anthropic Claude integration"""

    def __init__(self, api_key: str, model: str = "claude-sonnet-4-20250514"):
        self.client = anthropic.AsyncAnthropic(api_key=api_key)
        self.model = model

    async def complete(self, messages, tools=None, max_tokens=4000):
        response = await self.client.messages.create(
            model=self.model,
            max_tokens=max_tokens,
            messages=messages,
            tools=tools if tools else []
        )

        return LLMResponse(
            content=response.content,
            stop_reason=response.stop_reason,
            usage=response.usage
        )

class OpenAIProvider(LLMProvider):
    """OpenAI GPT integration"""

    def __init__(self, api_key: str, model: str = "gpt-4-turbo-preview"):
        self.client = openai.AsyncOpenAI(api_key=api_key)
        self.model = model

    async def complete(self, messages, tools=None, max_tokens=4000):
        response = await self.client.chat.completions.create(
            model=self.model,
            messages=messages,
```

```
        tools=tools if tools else [],
        max_tokens=max_tokens
    )

    return LLMResponse(
        content=response.choices[0].message.content,
        tool_calls=response.choices[0].message.tool_calls,
        usage=response.usage
    )
```

6.2 Prompt Templates

python

```
AGENT_SYSTEM_PROMPT = """You are an expert gaming companion AI assistant with deep knowledge of competitive ga

Your role is to help players improve their gameplay through data-driven analysis and actionable advice.

Available Tools:
{tool_descriptions}

When analyzing data:
1. Look for concrete patterns and trends
2. Provide specific, actionable recommendations
3. Back up claims with data
4. Consider the player's skill level and goals

Output Format:
- Use clear, structured reasoning
- Cite data sources when making claims
- Provide confidence levels for recommendations
"""

MATCH_ANALYSIS_PROMPT = """Analyze the player's recent matches and provide a comprehensive performance report.

Player Context:
- Game: {game}
- Rank: {rank}
- Main Role: {role}
- Recent Matches: {num_matches}

Tasks:
1. Fetch and analyze recent match data
2. Calculate key performance metrics
3. Identify strengths and weaknesses
4. Compare to typical performance at this rank
5. Provide 3-5 specific improvement recommendations

Focus on actionable insights that will help the player rank up."""
```

---

7. Error Handling & Resilience

7.1 Retry Logic

python

```python
class RetryHandler:
    """
    Handle retries for failed tool executions
    """

    async def execute_with_retry(
        self,
        tool: Tool,
        params: dict,
        max_retries: int = 3,
        backoff_factor: float = 2.0
    ) -> ToolResult:
        """
        Execute tool with exponential backoff retry
        """
        last_error = None

        for attempt in range(max_retries):
            try:
                result = await tool.execute(**params)

                if result.success:
                    return result

                # If tool returned error, consider if we should retry
                if not self.should_retry(result.error):
                    return result

                last_error = result.error

            except Exception as e:
                last_error = str(e)

            # Wait before retry
            if attempt < max_retries - 1:
                wait_time = backoff_factor ** attempt
                await asyncio.sleep(wait_time)

        # All retries exhausted
        return ToolResult(
            success=False,
            error=f"Failed after {max_retries} attempts: {last_error}",
            data={}
```

```
    )

def should_retry(self, error: str) -> bool:
    """Determine if error is retriable"""
    retriable_errors = [
        "rate_limit",
        "timeout",
        "connection_error",
        "503",
        "502"
    ]
    return any(e in error.lower() for e in retriable_errors)
```

7.2 Fallback Mechanisms

python

```python
class FallbackHandler:
    """
    Provide fallback options when agent fails
    """

    async def handle_agent_failure(
        self,
        task_state: TaskState,
        error: Exception
    ) -> FallbackResult:
        """
        Generate best possible response despite failure
        """
        # Try to use partial results
        if task_state.steps:
            partial_data = self.extract_partial_data(task_state)

            if partial_data:
                return FallbackResult(
                    type="partial_success",
                    data=partial_data,
                    message="Completed analysis with available data"
                )

        # Fall back to cached similar analysis
        cached_result = await self.get_similar_cached_result(task_state)
        if cached_result:
            return FallbackResult(
                type="cached_fallback",
                data=cached_result,
                message="Using recent similar analysis"
            )

        # Last resort: Generic advice
        return FallbackResult(
            type="generic_fallback",
            data=await self.generate_generic_advice(task_state),
            message="Providing general guidance"
        )
```

7.3 Circuit Breaker

python

```python
class CircuitBreaker:
    """
    Prevent cascading failures from unreliable tools
    """

    def __init__(
        self,
        failure_threshold: int = 5,
        reset_timeout: int = 60
    ):
        self.failure_threshold = failure_threshold
        self.reset_timeout = reset_timeout
        self.failures = {}
        self.open_circuits = {}

    async def call(self, tool_name: str, func, *args, **kwargs):
        """Execute function with circuit breaker protection"""

        # Check if circuit is open
        if self.is_open(tool_name):
            if self.should_attempt_reset(tool_name):
                # Try to reset circuit
                self.open_circuits[tool_name] = "half_open"
            else:
                raise CircuitBreakerOpen(f"Circuit open for {tool_name}")

        try:
            result = await func(*args, **kwargs)

            # Success - reset failure count
            self.record_success(tool_name)
            return result

        except Exception as e:
            # Failure - increment counter
            self.record_failure(tool_name)

            if self.should_open_circuit(tool_name):
                self.open_circuit(tool_name)

            raise e

    def record_failure(self, tool_name: str):
```

```
    """Record tool failure"""
    if tool_name not in self.failures:
        self.failures[tool_name] = []

    self.failures[tool_name].append(datetime.now())

    # Remove old failures outside window
    cutoff = datetime.now() - timedelta(seconds=self.reset_timeout)
    self.failures[tool_name] = [
        f for f in self.failures[tool_name] if f > cutoff
    ]
```

---

8. Performance Optimization

8.1 Caching Strategy

python

```python
class AgentCache:
    """
    Multi-level caching for agent results
    """

    def __init__(self, redis_client, ttl_config):
        self.redis = redis_client
        self.ttl_config = ttl_config
        self.local_cache = {}

    async def get_or_execute(
        self,
        cache_key: str,
        execute_func,
        cache_level: str = "standard"
    ):
        """
        Get from cache or execute and cache
        """
        # Try local cache first (in-memory)
        if cache_key in self.local_cache:
            return self.local_cache[cache_key]

        # Try Redis cache
        cached = await self.redis.get(cache_key)
        if cached:
            result = json.loads(cached)
            self.local_cache[cache_key] = result
            return result

        # Execute and cache
        result = await execute_func()

        # Cache with appropriate TTL
        ttl = self.ttl_config[cache_level]
        await self.redis.setex(
            cache_key,
            ttl,
            json.dumps(result)
        )

        self.local_cache[cache_key] = result
        return result
```

```
# TTL Configuration
CACHE_TTL_CONFIG = {
    "match_data": 300,      # 5 minutes
    "player_stats": 600,    # 10 minutes
    "meta_analysis": 3600,  # 1 hour
    "game_knowledge": 86400 # 24 hours
}
```

8.2 Parallel Tool Execution

python

```python
class ParallelExecutor:
    """
    Execute independent tools in parallel
    """

    async def execute_parallel(
        self,
        tool_calls: list[ToolCall]
    ) -> list[ToolResult]:
        """
        Execute multiple tool calls concurrently
        """
        # Group into dependency levels
        execution_graph = self.build_dependency_graph(tool_calls)

        results = {}

        # Execute each level in parallel
        for level in execution_graph.levels:
            tasks = []

            for tool_call in level:
                # Resolve dependencies
                params = self.resolve_params(
                    tool_call.params,
                    results
                )

                task = self.execute_tool(
                    tool_call.tool_name,
                    params
                )
                tasks.append((tool_call.id, task))

            # Wait for all tasks in this level
            level_results = await asyncio.gather(
                *[task for _, task in tasks],
                return_exceptions=True
            )

            # Store results
            for (call_id, _), result in zip(tasks, level_results):
                results[call_id] = result
```

```
    return results
```

8.3 Token Usage Optimization

python

```python
class TokenOptimizer:
    """
    Optimize LLM token usage
    """

    def optimize_context(
        self,
        messages: list,
        max_context_tokens: int = 100000
    ) -> list:
        """
        Trim context to fit within token budget
        """
        # Calculate token count
        total_tokens = sum(
            self.count_tokens(msg["content"])
            for msg in messages
        )

        if total_tokens <= max_context_tokens:
            return messages

        # Keep system message and recent messages
        optimized = [messages[0]]  # System message

        # Add recent messages within budget
        remaining_budget = max_context_tokens - self.count_tokens(
            messages[0]["content"]
        )

        for msg in reversed(messages[1:]):
            msg_tokens = self.count_tokens(msg["content"])

            if msg_tokens <= remaining_budget:
                optimized.insert(1, msg)
                remaining_budget -= msg_tokens
            else:
                break

        return optimized

    def summarize_tool_results(
        self,
```

```
    tool_results: list[ToolResult]
) -> str:
    """
    Create concise summary of tool results
    """
    summaries = []

    for result in tool_results:
        if result.success:
            # Extract key information only
            summary = self.extract_key_info(result.data)
            summaries.append(summary)

    return "\n".join(summaries)
```

---

9. Monitoring & Observability

9.1 Metrics Collection

python

```python
class AgentMetrics:
    """
    Collect and report agent performance metrics
    """

    def __init__(self, metrics_client):
        self.metrics = metrics_client

    async def record_task_execution(
        self,
        task_type: str,
        duration: float,
        success: bool,
        tool_calls: int,
        tokens_used: int
    ):
        """Record metrics for task execution"""

        self.metrics.histogram(
            "agent.task.duration",
            duration,
            tags={"task_type": task_type, "success": success}
        )

        self.metrics.counter(
            "agent.task.count",
            tags={"task_type": task_type, "success": success}
        )

        self.metrics.gauge(
            "agent.tool_calls",
            tool_calls,
            tags={"task_type": task_type}
        )

        self.metrics.gauge(
            "agent.tokens_used",
            tokens_used,
            tags={"task_type": task_type}
        )
```

9.2 Logging Strategy

python

```python
import structlog

logger = structlog.get_logger()

class AgentLogger:
    """
    Structured logging for agent operations
    """

    async def log_task_start(self, task_state: TaskState):
        logger.info(
            "agent.task.started",
            task_id=task_state.task_id,
            user_id=task_state.user_id,
            task_type=task_state.type
        )

    async def log_tool_execution(
        self,
        task_id: str,
        tool_name: str,
        params: dict,
        result: ToolResult,
        duration: float
    ):
        logger.info(
            "agent.tool.executed",
            task_id=task_id,
            tool_name=tool_name,
            success=result.success,
            duration_ms=duration * 1000,
            error=result.error if not result.success else None
        )

    async def log_task_completion(
        self,
        task_state: TaskState,
        total_duration: float
    ):
        logger.info(
            "agent.task.completed",
            task_id=task_state.task_id,
            status=task_state.status,
```

```
            num_steps=len(task_state.steps),
            total_duration_s=total_duration
        )
```

---

10. API Contracts

10.1 Request Format

json

```
{
  "task_type": "match_analysis",
  "user_id": "user_123",
  "parameters": {
    "game": "league_of_legends",
    "num_matches": 20,
    "focus_areas": ["laning", "teamfighting"]
  },
  "context": {
    "player_rank": "Gold 2",
    "main_role": "ADC",
    "timezone": "America/New_York"
  },
  "options": {
    "timeout": 30,
    "priority": "normal",
    "webhook_url": "https://api.example.com/webhook"
  }
}
```

10.2 Response Format

json

```
{
 "task_id": "task_abc123",
 "status": "completed",
 "result": {
  "summary": "Analysis of 20 recent matches...",
  "data": {
   "overall_stats": {
    "win_rate": 0.55,
    "avg_kda": 3.2,
    "matches_analyzed": 20
   },
   "strengths": [
    "Consistent CS in laning phase",
    "Good positioning in teamfights"
   ],
   "weaknesses": [
    "Low vision score",
    "Dies early in 40% of losses"
   ],
   "recommendations": [
    {
     "priority": "high",
     "title": "Improve warding",
     "description": "Place 2-3 more wards per game...",
     "expected_impact": "Reduce deaths by 15-20%"
    }
   ]
  },
  "sources": [
   {
    "type": "game_api",
    "data_points": 20
   },
   {
    "type": "web_search",
    "urls": ["https://..."]
   }
  ]
 },
 "execution_stats": {
  "duration_seconds": 12.5,
  "tool_calls": 6,
  "tokens_used": 15000,
```

```
    "api_calls": 3
  },
  "created_at": "2025-11-07T10:30:00Z",
  "completed_at": "2025-11-07T10:30:12Z"
}
```

10.3 WebSocket Updates

For long-running tasks, provide real-time updates:

json

```
{
  "task_id": "task_abc123",
  "event": "step_completed",
  "step": {
    "name": "fetch_matches",
    "status": "completed",
    "message": "Fetched 20 matches from Riot API"
  },
  "progress": 0.33,
  "timestamp": "2025-11-07T10:30:05Z"
}
```

---

11. Cost Management

11.1 Cost Tracking

python

```python
class CostTracker:
    """
    Track and manage API costs
    """

    # Cost per 1M tokens (example prices)
    COSTS = {
        "claude-opus-4": {
            "input": 15.0,
            "output": 75.0
        },
        "claude-sonnet-4": {
            "input": 3.0,
            "output": 15.0
        },
        "gpt-4-turbo": {
            "input": 10.0,
            "output": 30.0
        }
    }

    async def calculate_cost(
        self,
        model: str,
        input_tokens: int,
        output_tokens: int
    ) -> float:
        """Calculate cost for LLM usage"""

        costs = self.COSTS.get(model, self.COSTS["claude-sonnet-4"])

        input_cost = (input_tokens / 1_000_000) * costs["input"]
        output_cost = (output_tokens / 1_000_000) * costs["output"]

        return input_cost + output_cost

    async def check_user_budget(
        self,
        user_id: str,
        estimated_cost: float
    ) -> bool:
        """Check if user has budget for operation"""
```

```
        user_usage = await self.get_user_usage(user_id)
        user_tier = await self.get_user_tier(user_id)

        monthly_limit = self.TIER_LIMITS[user_tier]

        return user_usage + estimated_cost <= monthly_limit
```

11.2 Tier Limits

python

```
TIER_LIMITS = {
    "free": {
        "monthly_cost_limit": 0.50,    # $0.50
        "queries_per_day": 5,
        "max_tool_calls": 5
    },
    "basic": {
        "monthly_cost_limit": 5.0,     # $5.00
        "queries_per_day": 50,
        "max_tool_calls": 10
    },
    "pro": {
        "monthly_cost_limit": 25.0,    # $25.00
        "queries_per_day": 500,
        "max_tool_calls": 20
    },
    "elite": {
        "monthly_cost_limit": 100.0,   # $100.00
        "queries_per_day": float('inf'),
        "max_tool_calls": float('inf')
    }
}
```

---

12. Security Considerations

12.1 Input Validation

python

```
class InputValidator:
    """
    Validate and sanitize user inputs
    """

    def validate_task_request(self, request: dict) -> tuple[bool, str]:
        """Validate incoming task request"""

        # Check required fields
        required = ["task_type", "user_id"]
        for field in required:
            if field not in request:
                return False, f"Missing required field: {field}"

        # Validate task type
        if request["task_type"] not in ALLOWED_TASK_TYPES:
            return False, "Invalid task type"

        # Sanitize parameters
        if "parameters" in request:
            sanitized = self.sanitize_params(request["parameters"])
            request["parameters"] = sanitized

        # Check for injection attempts
        if self.contains_malicious_content(request):
            return False, "Malicious content detected"

        return True, "Valid"

    def sanitize_params(self, params: dict) -> dict:
        """Remove potentially dangerous content"""
        sanitized = {}

        for key, value in params.items():
            if isinstance(value, str):
                # Remove script tags, SQL, etc.
                value = self.remove_dangerous_patterns(value)

            sanitized[key] = value

        return sanitized
```

12.2 Rate Limiting

python

```python
class RateLimiter:
    """
    Rate limiting for agent requests
    """

    def __init__(self, redis_client):
        self.redis = redis_client

    async def check_rate_limit(
        self,
        user_id: str,
        tier: str
    ) -> tuple[bool, int]:
        """
        Check if user is within rate limits
        Returns (allowed, remaining_requests)
        """
        key = f"rate_limit:{user_id}:{date.today()}"

        current = await self.redis.get(key)
        current = int(current) if current else 0

        limit = TIER_LIMITS[tier]["queries_per_day"]

        if current >= limit:
            return False, 0

        # Increment counter
        await self.redis.incr(key)
        await self.redis.expire(key, 86400)  # 24 hours

        return True, limit - current - 1
```

---

13. Testing Strategy

13.1 Unit Tests

python

```python
import pytest

class TestGamingAgent:
    """Unit tests for agent core functionality"""

    @pytest.mark.asyncio
    async def test_agent_reasoning(self):
        """Test agent reasoning loop"""
        agent = GamingAgent(
            llm_client=MockLLM(),
            tools=MockToolRegistry()
        )

        result = await agent.execute_task(
            "Analyze player performance",
            context={"player_id": "test_123"}
        )

        assert result.success
        assert len(result.steps) > 0

    @pytest.mark.asyncio
    async def test_tool_execution(self):
        """Test individual tool execution"""
        tool = GameAPITool()

        result = await tool.execute(
            game="lol",
            endpoint="match_history",
            player_id="test_player"
        )

        assert result.success
        assert "matches" in result.data

    @pytest.mark.asyncio
    async def test_error_handling(self):
        """Test agent handles errors gracefully"""
        agent = GamingAgent(
            llm_client=MockLLM(),
            tools=FailingToolRegistry()
        )
```

```
    result = await agent.execute_task(
        "Test error handling",
        context={}
    )

    # Should return fallback result
    assert result.type == "partial_success"
```

13.2 Integration Tests

python

```
class TestAgentIntegration:
    """Integration tests with real APIs"""

    @pytest.mark.integration
    @pytest.mark.asyncio
    async def test_match_analysis_workflow(self):
        """Test full match analysis workflow"""

        # Setup
        agent = create_production_agent()

        # Execute
        result = await agent.execute_task(
            task_type="match_analysis",
            user_context={
                "user_id": "test_user",
                "game": "lol",
                "player_id": "real_player_id"
            }
        )

        # Verify
        assert result.success
        assert "overall_stats" in result.data
        assert "recommendations" in result.data
        assert len(result.recommendations) >= 3
```

13.3 Load Testing

python

```
import locust

class AgentUser(HttpUser):
    """Load test user simulation"""

    @task
    def quick_analysis(self):
        """Simulate quick analysis request"""
        self.client.post("/api/agent/analyze", json={
            "task_type": "quick_stats",
            "user_id": "load_test_user",
            "parameters": {
                "game": "lol",
                "num_matches": 5
            }
        })

    @task(2)
    def deep_analysis(self):
        """Simulate deep analysis request (less frequent)"""
        self.client.post("/api/agent/analyze", json={
            "task_type": "match_analysis",
            "user_id": "load_test_user",
            "parameters": {
                "game": "lol",
                "num_matches": 20
            }
        })
```

---

14. Deployment Considerations

14.1 Infrastructure Requirements

Minimum Production Setup:

- **API Servers**: 2-4 instances (auto-scaling)
  - 4 vCPU, 8GB RAM each
  - Load balanced
- **Worker Instances**: 2-4 instances (for agent tasks)
  - 8 vCPU, 16GB RAM each

    - Task queue based scaling

- **Redis**: Managed service or cluster
    - 8GB memory minimum
    - Persistence enabled

- **PostgreSQL**: Managed database
    - 4 vCPU, 16GB RAM
    - Read replicas for analytics

- **Message Queue**: RabbitMQ or AWS SQS
    - For async task distribution

14.2 Deployment Pipeline

yaml

```yaml
# .github/workflows/deploy-agent.yml
name: Deploy Agent Service

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: Run Tests
        run: |
          pytest tests/

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Build Docker Image
        run: |
          docker build -t gaming-companion-agent:${{ github.sha }} .

      - name: Push to Registry
        run: |
          docker push registry.example.com/agent:${{ github.sha }}

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Production
        run: |
          kubectl set image deployment/agent \
            agent=registry.example.com/agent:${{ github.sha }}
```

15. Future Enhancements

Phase 1 (Months 1-3)

- [ ] Basic agent with 3 core tools
- [ ] Simple match analysis workflow
- [ ] Cost tracking and limits

Phase 2 (Months 4-6)

- [ ] Advanced tools (video analysis, voice processing)
- [ ] Multi-agent collaboration
- [ ] Predictive analytics

Phase 3 (Months 7-9)

- [ ] Self-improving agents (learn from feedback)
- [ ] Community knowledge contribution
- [ ] Real-time coaching during gameplay

Phase 4 (Months 10-12)

- [ ] Cross-game meta-learning
- [ ] Tournament preparation suite
- [ ] Pro player analysis integration

---

16. Appendix

16.1 Glossary

- **Agent**: Autonomous AI system that uses tools and reasoning
- **Tool**: Discrete capability the agent can invoke
- **Workflow**: Predefined sequence of agent actions
- **ReAct**: Reasoning + Acting pattern for agents
- **Circuit Breaker**: Pattern to prevent cascading failures

16.2 References

- LangGraph Documentation
- Anthropic Tool Use Guide

- OpenAI Function Calling
- Riot Games API

End of Technical Specification